
Report on Assignment 1 for Reinforcement Learning: Deep Q-learning

Sam Beckers¹

Abstract

We present a study of Deep Q-learning (DQN) in Reinforcement Learning. We implemented our own DQN algorithm in Gymnasium environments and performed an ablation study on several hyperparameters. We also implemented Experience Replay and a Target Network and compared this to Naive DQN. Moreover, next to the `CartPole` environment, we experimented with the `LunarLander` environment and compared Dueling DQN to Naive DQN.

1. Introduction

In Reinforcement Learning (RL), we often have to deal with large datasets. With tabular Q-learning methods, we wish to find the best policy given the state-action value function $Q(s, a)$, where s is a state and a an action, based on a reward system. At some point, the dimensionality of the dataset can become so large that we cannot simply store this anymore. In this case, we have to turn to *function approximation*, which is given by $Q_\theta(s, a) \approx Q(s, a)$, where θ is the approximation identifier.

Technically, any environment you can dream of can be an environment to solve with Q-learning. Hence, we need to be prepared to approximate high-dimension functions. This can be achieved with Deep Q-learning (DQN), which employs deep neural networks in Q-learning to estimate state values and solve the RL problem (Mnih et al., 2013; Van Hasselt et al., 2016). Although DQN is not perfect at approximating functions (Van Hasselt et al., 2016), it can solve these high-dimensionality cases, and can also be improved upon by e.g. Double DQN or Dueling DQN (Hessel et al., 2017).

In this report, we discuss our implementation of DQN to solve OpenAI's Gymnasium environments (Towers et al., 2024), specifically the `CartPole` environment (and `LunarLander` as a bonus). First, we discuss some theory

^{*}Equal contribution ¹Leiden Observatory, University of Leiden, Leiden, The Netherlands. Correspondence to: Sam Beckers <beckers@strw.leidenuniv.nl>.

behind the DQN algorithm in section 2. In section 3, we explain the experiments we performed using DQN and discuss the results of those studies. In section 4, we have a general discussion concerning our implementation and experiments. Lastly, we state our conclusions in section 5.

2. Theory

2.1. Neural Networks

A Neural Network (NN), or better, an Artificial Neural Network (ANN), is based on the neuron structure within animal biology (Ranganathan et al., 2019). Artificial nodes are connected through each other in layers which contain network parameters with which a mathematical function can be approximated.

We chose to work with a simple NN with 3 layers made with the `pytorch` deep learning library (Paszke et al., 2019), where we can customize the number of neurons and the dimension of state/action space. Although we could have chosen for more layers, we opted for 3 layers for the sake of lower computation times. We also experimented with a Convolutional NN (CNN), however, this did not yield significant improvement and was not worth the extra complexity.

2.2. DQN

Q-learning is based on the Bellman equation for Dynamic Programming, which aims to find the value function based on the current state, next state and a discount factor in a Markov Decision Process (MDP). In Q-learning, the "value function", $Q(s, a)$ is updated iteratively with

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

where α is the learning rate, r the reward and γ the discount factor.

In Deep Q-learning, this works slightly differently, as instead of returning $Q(s, a)$ we return $Q_\theta(s, a)$, since all the Q-values have been approximated by the parameters of the NN, which are denoted by the approximation identifier θ . We can get the Q-values of the current and next state from the NN. To optimize the network, we have to calculate a loss function. Generally, the loss function calculates the difference between observed and expected values. Here, we calculate the difference between the calculated $Q_\theta(s, a)$ and

a target $Q(s, a)$, call this G' , which can also be called the back-up function. G' is given by

$$G' = r + \gamma \cdot \max_{a'} Q(s', a') \quad (2)$$

In tabular Q-learning, we would have updated $Q(s, a)$ with this back-up function for every timestep. These update rules cannot be used in DQN, as we have to constantly update the NN parameters and cannot store every state-action pair in a table. Instead, the Mean-Squared Error (MSE) loss is often used to update the NN here, which is given, in the context of DQN, by

$$\text{loss}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (Q(s, a) - G')^2 \quad (3)$$

where N is the number of values. We can optimize the NN with this loss using an Adaptive Moment (Adam) optimizer (PyTorch, 2024).

2.3. Experience Replay

As we sometimes noticed in the experiments, DQN tends to forget what it learned in previous episodes. To resolve this, we can use a replay buffer to store those experiences and use these experiences during updates, which is called Experience Replay (ER) (Schaul et al., 2015). We can create a buffer of a certain length and store experiences (s, a, r, s') values in this buffer. Whenever we call on ER, we can sample a random batch from the memory buffer, and pass it to the NN. This is theoretically more stable and accurate than Naive DQN.

2.4. Target Network

We can also use a Target Network (TN) to improve upon Naive DQN. Since we use the bootstrapped Q-values from the NN to compute G' , a bias can arise in the MSE loss (Fan et al., 2020). A TN uses the same architecture and is updated with the state parameters from the NN at an interval of certain episodes and is used to compute $Q(s', a')$, which in turn is then used in computing an unbiased MSE loss. We can also feed the experiences in memory to the TN instead to get a combined ER + TN DQN algorithm. We show this in Algorithm 1.

2.5. Dueling DQN (bonus)

In (Hessel et al., 2017), we see that there are many variations of DQN that can improve performance and stability. Dueling DQN is one of them, and works with the following equation (Wang et al., 2015):

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{\mathcal{A}} \sum_{a'} |A(s, a')| \quad (4)$$

Algorithm 1 DQN with ER and TN

Input: environment, NN, TN, exploration parameter, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, total *budget*

while *budget* **do**

$s = s_0$ {Sample initial state}

while s is not terminal **do**

$a \sim \pi(a|s)$ {Sample action}

$r, s' \sim p(r, s'|s, a)$ {Simulate environment}

if ER **then**

memory $\leftarrow s, a, r, s'$ {Send experience to replay buffer}

end if

if TN **then**

$Q(s, a), Q(s', a') \leftarrow TN(s), TN(s')$

else

$Q(s, a), Q(s', a') \leftarrow NN(s), NN(s')$

end if

$G' = r + \gamma \cdot \max_{a'} Q(s', a')$

loss = MSE($Q(s, a), G'$)

NN $\leftarrow$ loss, optimize

$s = s'$

end while

end while

where $V(s)$ is the value function, $A(s, a)$ the advantage function and $\mathcal{A}$ a normalization scalar of the advantage stream. The advantage stream assesses how much of an advantage an action has over the mean actions during training. Hence, dueling DQN can better assess which actions help in solving the environment. In our implementation, we only need to replace our simple NN with an NN with a dueling architecture to enable dueling DQN.

3. Experiments

3.1. Environments

The RL environments used were developed in OpenAI's Gymnasium library, and are maintained by the Farama Foundation. `CartPole` is known as one of the classic RL problems. The idea of this environment is simple: a pole balances on a cart and the goal is to keep the pole on the cart. You can either push the cart to the left or right. A little more complex environment is the `LunarLander`, which attempts to land a lunar lander between two flagpoles using the rocket engines. Here, the rocket engines can be either on or off. When loading in the environments, we can access the observation and action space, and perform steps.

3.2. Ablation study

3.2.1. LEARNING RATE

The learning rate α can substantially affect the way DQN learns, as seen in Figure 1.

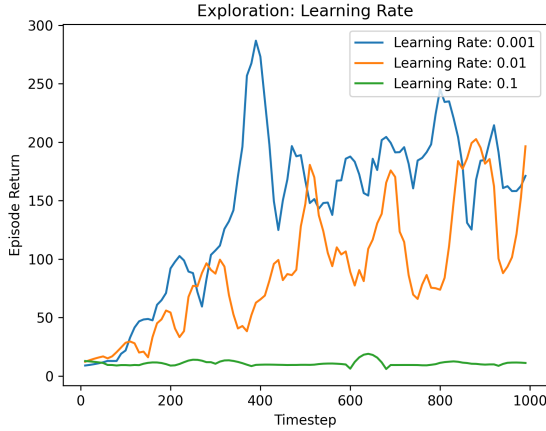


Figure 1. Episode return as a function of episodes (timesteps) for learning rate $\alpha = [0.001, 0.01, 0.1]$. The lowest α has the highest episode return whilst remaining quite stable. Similarly, the medium α has a bit more stability with generally lower episode return, whilst the high α demonstrates that DQN does not learn when the learning rate is too high.

3.2.2. NETWORK SIZE

The network size refers to the amount of neurons of the NN. This influences the function approximation and computation times of DQN. See Figure 2.

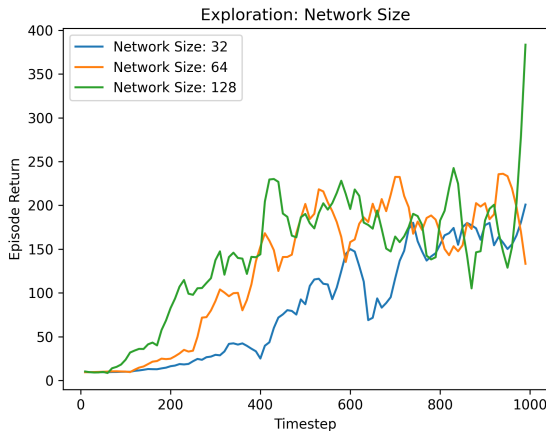


Figure 2. Episode return as a function of episodes (timesteps) for network size of $[32, 64, 128]$. Here, the highest network size dominates in the episode return over the entire interval, whilst they are all similar in terms of stability. Hence, a size of 128 is preferable even though it may come at the cost of slightly higher computation time.

3.2.3. UPDATE-TO-DATA RATIO

The Update-to-Data Ratio (UTDR) (Dorka et al., 2023) affects how often the agent is updated. E.g. with $UTDR = 1$, the agent is updated after every environment step. The effects on how DQN learns are shown in Figure 3

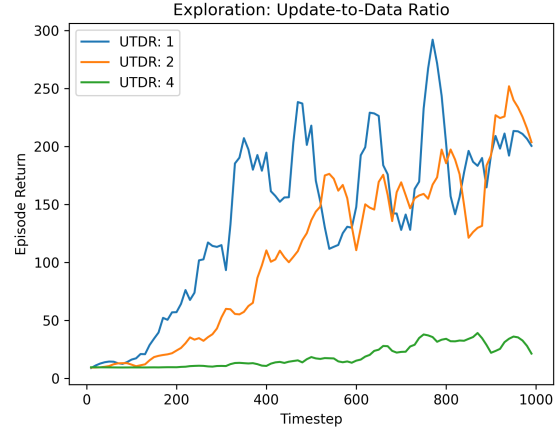


Figure 3. Episode return as a function of episodes (timesteps) for UTDR of $[1, 2, 4]$. The low has the highest episode return whilst being less stable than the medium. A high UTDR results in the agent not learning.

3.2.4. EXPLORATION FACTOR

For this experiment, we opted to use the ϵ -greedy policy, and vary ϵ as seen in Figure 4.

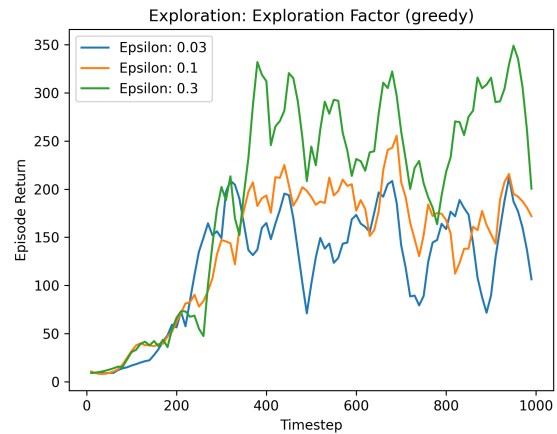


Figure 4. Episode return as a function of episodes (timesteps) for $\epsilon = [0.03, 0.1, 0.3]$. The lower ϵ is quite unstable and does not reach desirable episode return. Whilst the higher ϵ has the highest episode return, it is significantly more unstable than the medium ϵ , and therefore we prefer the medium ϵ .

3.3. Configurations

With ER and TN we can explore different DQN configurations. These are: Naive - Only TN - Only ER - TN &

ER. We run experiments on all configurations with fixed hyperparameters, based on the above ablation study. See Figure 5.

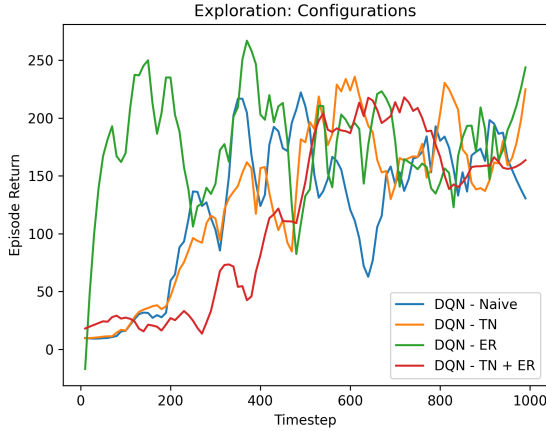


Figure 5. Episode return as a function of episodes (timesteps) for different DQN configurations. ER DQN has the highest episode return but is quite unstable from the start. TN DQN shows similar behavior to Naive DQN. TN+ER DQN needs more episodes to get to comparable episode returns, but is quite stable otherwise. Compared to Naive DQN, we can classify this as an improvement, albeit not substantial, in the performance of the agent.

3.4. Lunar Lander (bonus)

We experimented with the `LunarLander` environment to see if it would work with our DQN implementation. We used the same hyperparameters as in the configurations experiment. The results are shown in Figure 6.

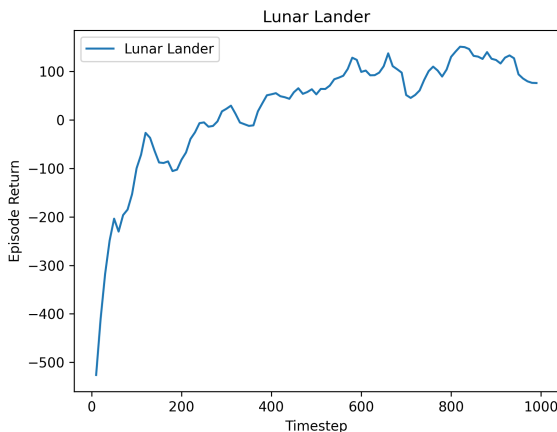


Figure 6. Episode return as a function of episodes (timesteps) for Naive DQN on the `LunarLander` environment. Although the episode return is negative at the start, eventually DQN also learns in this environment and has positive episode return.

3.5. Dueling DQN (bonus)

We also experimented with Dueling DQN to see how it would compare to Naive DQN. We used the same hyperparameters as in the configurations experiment. The results are shown in Figure 7.

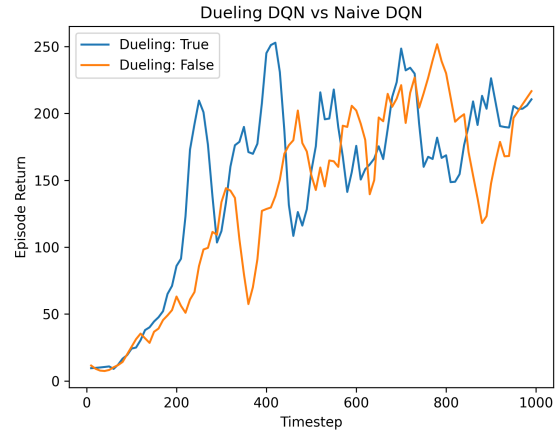


Figure 7. Episode return as a function of episodes (timesteps) for Dueling DQN compared to Naive DQN. As expected from (Hessel et al., 2017), Dueling DQN performs better than Naive DQN.

4. Discussion

Generally, we believe our DQN implementation works well. The ablation study consistently showed how hyperparameters can affect the training of the DQN agent, with considerations made both on episode return and stability. The results are consistent with what is intuitively understood about these hyperparameters, which suggests they are correctly implemented in the algorithm. For the ER and TN experiments, we would have expected to see a more substantial difference with Naive DQN. We suspect that the additional operations may have had an impact on the numerical stability and performance of the CPU over time. Moreover, we are satisfied with the DQN agent being able to train on the `LunarLander` environment as well, and that Dueling DQN outperforms Naive DQN.

The weakness of our experiments was that we had to rely on CPU calculations on a STRW server instead of LIACS GPUs (access was denied). Therefore, we could only run 1000 episodes, which show enough of a trend to draw conclusions on our DQN implementation, but are not ideal compared to 10^6 episodes. We tried to adapt our PyTorch code to handle vectorized `CartPole` with no success; any tips/feedback on this is welcome.

5. Conclusion

In conclusion, we implemented our own DQN algorithm in Gymnasium environments. We did an ablation study on sev-

eral hyperparameters, including the learning rate, network size, UTDR and exploration factor, which showed consistent results. We implemented ER and TN and compared that to Naive DQN, which did not show a significant increase in performance. Moreover, we successfully experimented with the LunarLander environment and outperformed Naive DQN with Dueling DQN. We would have liked to see our DQN agent run on GPUs and vectorize our code, but we are satisfied with the results that we obtained from CPU calculations.

References

- Dorka, N., Welschhold, T., and Burgard, W. Dynamic Update-to-Data Ratio: minimizing world model overfitting, 3 2023. URL <https://arxiv.org/abs/2303.10144>.
- Fan, J., Wang, Z., Xie, Y., and Yang, Z. A theoretical analysis of deep q-learning. In Bayen, A. M., Jadbabaie, A., Pappas, G., Parrilo, P. A., Recht, B., Tomlin, C., and Zeilinger, M. (eds.), *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pp. 486–489. PMLR, 10–11 Jun 2020. URL <https://proceedings.mlr.press/v120/yang20a.html>.
- Hessel, M., Modayil, J., Hado, V. H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: combining improvements in deep reinforcement learning, 10 2017. URL <https://arxiv.org/abs/1710.02298>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. Playing Atari with Deep Reinforcement Learning, 12 2013. URL <https://arxiv.org/abs/1312.5602>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 12 2019. URL <https://arxiv.org/abs/1912.01703>.
- PyTorch. Adam — PyTorch 2.6 documentation, 2024. URL <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html>.
- Ranganathan, S., Gribskov, M., Nakai, K., and Schönbach, C. (eds.). *Encyclopedia of bioinformatics and computational biology*, volume 1-3. Elsevier, Netherlands, 2019. ISBN 9780128114148.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., Gianluca, D. C., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., Kg, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., and Younis, O. G. Gymnasium: a standard interface for reinforcement learning environments, 7 2024. URL <https://arxiv.org/abs/2407.17032>.
- Van Hasselt, H., Guez, A., and Silver, D. Deep Reinforcement Learning with Double Q-Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 3 2016. doi: 10.1609/aaai.v30i1.10295. URL <https://doi.org/10.1609/aaai.v30i1.10295>.
- Wang, Z., Schaul, T., Hessel, M., Hado, V. H., Lanctot, M., and Nando, D. F. Dueling network architectures for deep reinforcement learning, 11 2015. URL <https://arxiv.org/abs/1511.06581>.